

ECA1422 – INDIVIDUAL SELF-REFLECTION & LEARNING EVIDENCE

AI-Enabled RTOS-Based Industrial Machine-Condition Monitoring

Student: G. Jaswanth

Register No.: 192472235

This report documents my individual design, analysis, simulation work, technical learning, challenges, evidence and future improvements. The numerical results are tied to the proposed motor/pump predictive-maintenance system and are identified as design/simulation results where hardware measurements are not yet available.

1. Introduction

I designed an AI-enabled RTOS-based embedded system for predictive maintenance of an industrial motor/pump. The system combines high-rate vibration acquisition, supporting temperature/current indicators, FreeRTOS task scheduling, interrupt-driven fault handling, a compact edge AI classifier and best-effort MQTT communication. My individual focus was to connect the application requirements to a deterministic task model, timing analysis, SystemC-based modeling approach and embedded-AI design.

2. My Individual Contribution

Activity	My Individual Work	Evidence
Industrial problem	Defined the motor/pump predictive-maintenance problem, requirements, constraints and safety objectives.	Technical report Sections 1–2
RTOS design	Designed SensorTask, FeatureTask, AITask, CommTask, HealthTask and the higher-priority FaultTask; assigned priorities, WCET, periods and deadlines.	Task table + FreeRTOS source
Scheduling	Calculated 33.17% nominal	RTA calculations + scheduling

	periodic utilization and fixed-priority response times; analysed overload behavior.	trace
Interrupt	Designed the vibration-threshold GPIO ISR and ISR-to-FaultTask binary semaphore path.	Interrupt design + timing budget
SystemC	Developed the timed process/event abstraction and scheduler model structure; cross-checked the same timing parameters with a reference discrete-event scheduler.	SystemC source + trace
AI/ML	Defined five features, synthetic 1,200-window dataset structure, depth-4 decision tree and held-out validation.	AI results + confusion matrix
Resources	Estimated stacks, queues, buffers, model storage, Flash and CPU utilization.	Memory/resource tables
Safety	Designed sensor timeout, AI overrun, network-loss, memory-exhaustion and watchdog responses.	Safety matrix
Optimization	Designed event-driven scheduling, communication batching, compact AI and low-power idle strategy.	Energy comparison

3. Learning in Embedded C

The assignment strengthened my understanding of modular Embedded C. Instead of treating the firmware as one large loop, I learned to separate acquisition, feature extraction, AI inference, communication, health monitoring and interrupt handling. I learned to use fixed-size buffers and explicit data structures where predictable memory behavior is important. I also learned that an ISR should perform only the minimum time-critical work, such as latching an event and signaling a task, rather than performing MQTT communication or long calculations. Debug-oriented design, return-code checks, stack-watermark monitoring and watchdog handling were identified as necessary additions for production firmware.

4. Learning in RTOS

I learned that real-time behavior depends on the interaction of period, WCET, deadline and priority. I used preemptive fixed-priority scheduling so that SensorTask and the emergency FaultTask can preempt lower-priority work. I learned to isolate network communication because network latency is variable and should not interfere with deterministic sensing or safety. Queues provide producer-consumer communication, mutexes protect shared resources, semaphores provide ISR-to-task signaling, and event flags represent system-wide conditions. Priority inheritance was included to reduce priority inversion when shared resources are required.

The calculated nominal periodic utilization is 33.17%, below the 70% design target. Response-time analysis gives 0.18 ms for SensorTask, 2.34 ms for FeatureTask, 7.74 ms for AITask, 17.54 ms for CommTask and 18.42 ms for HealthTask, all below their respective deadlines. These calculations helped me understand why a task set can be considered schedulable under stated assumptions.

5. Learning in SystemC

SystemC modeling helped me represent the design as concurrent timed processes rather than only as source-code functions. I modeled task releases, execution durations, priority selection, preemption and deadline checking as discrete events. The SystemC source uses the same WCET, period and priority values as the RTOS design, which makes the model traceable to the analytical calculations. A reference discrete-event scheduler was used to cross-check the 0–100 ms trace and confirm zero nominal deadline misses. A SystemC toolchain is required to independently compile and rerun the model.

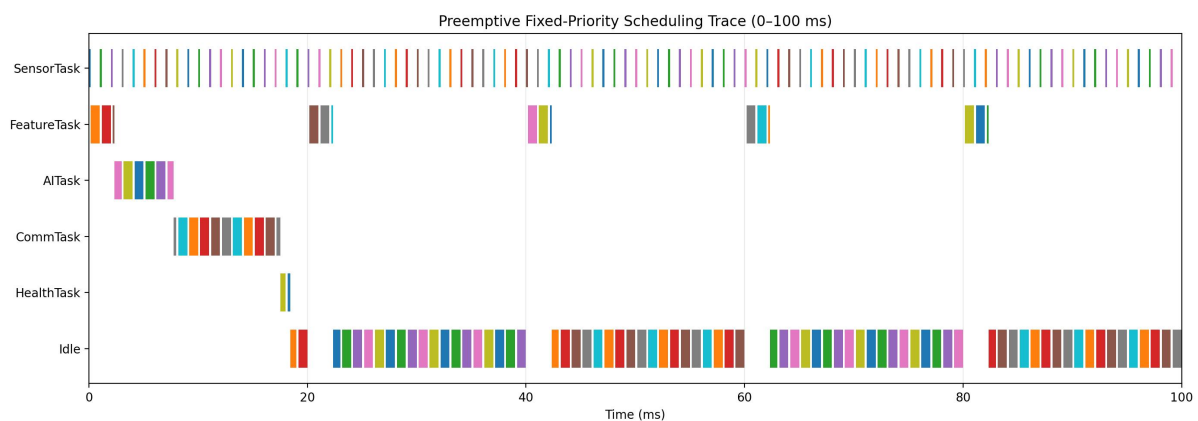


Figure 1. Representative preemptive fixed-priority scheduling trace used as individual simulation evidence.

6. Learning in AI/ML

I learned that an embedded AI model must be selected using both classification performance and system constraints. The design uses RMS acceleration, kurtosis, crest factor, dominant frequency and temperature as engineered features. A depth-4 decision tree was selected because it has a small number of comparisons, predictable inference behavior and an estimated model size below 4 KB. The synthetic dataset contains 1,200 windows, split into 960 training and 240 held-out test samples. The held-out accuracy is 99.17%, with 80/80 Normal, 79/80 Misalignment and 79/80 Bearing Fault correctly classified. This is synthetic validation only; field accuracy requires real labelled machine data.

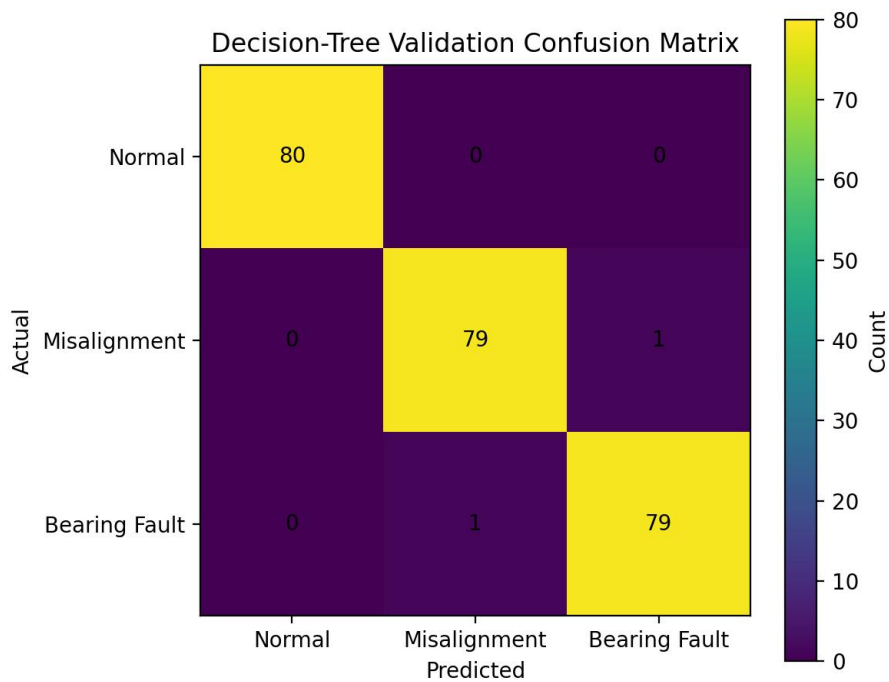


Figure 2. Confusion matrix for the held-out synthetic AI validation set.

7. Challenges and Solutions

Challenge	Cause	Solution	Result	Learning
Balancing RTOS priorities	High-rate sensing, feature extraction and AI inference compete for CPU time.	Assigned priorities by real-time urgency and verified response times with fixed-priority analysis.	All nominal task deadlines satisfied.	Priority should reflect timing criticality, not code size.
Preventing	MQTT	Separated	Safety-critical	Non-deterministic

network interference	transactions have variable latency.	communication using a queue and assigned CommTask low priority.	tasks remain independent of network delay.	services should not block deterministic tasks.
Making AI suitable for an MCU	Large models can consume memory and create less predictable execution.	Selected a depth-4 decision tree with five features.	Small model with 4.5 ms target WCET.	Edge AI is a trade-off among accuracy, latency and memory.
Handling urgent vibration faults	Polling can delay detection of a threshold event.	Used a GPIO interrupt, short ISR and binary semaphore to wake FaultTask.	Fast fault path without lengthy ISR work.	Interrupt handlers must be minimal and deterministic.

8. Evidence of Individual Work

The following evidence is included or prepared as part of this submission:

- Embedded C/FreeRTOS task structure and ISR design
- RTOS task priority/WCET/period/deadline table
- CPU-utilization and response-time calculations
- SystemC scheduler model source
- Reference scheduling trace and simulation output
- AI training/evaluation definition and confusion matrix
- Memory and resource analysis
- Safety/failure-condition matrix
- Energy optimization comparison
- Architecture diagram and technical report calculations

The supplementary simulation bundle contains the SystemC source, reference scheduler script and generated analysis figures. Version-control screenshots may be added if a repository is used.

9. Skills Gained

Technical Skills	Engineering Skills
Embedded C modular programming	Problem identification and requirements analysis
FreeRTOS tasks and preemption	System architecture and trade-off decisions

Queues, mutexes, semaphores and event flags	Real-time performance evaluation
Interrupt-driven programming	Safety and failure analysis
SystemC timed-process modeling	Technical documentation
AI feature preparation and decision-tree evaluation	Resource and energy optimization
Scheduling and response-time analysis	Evidence-based engineering judgment
Debugging and memory management	Design verification and feasibility assessment

10. Future Improvements

Improvement	Why Required	Implementation Approach
Real industrial dataset	Synthetic data cannot establish field accuracy or false-alarm rate.	Collect labelled vibration data across machine speeds, loads and fault severities; validate across separate machines.
Measured WCET and interrupt latency	Current values are design budgets.	Use ESP32-S3 runtime statistics and GPIO/logic-analyzer timestamps.
AI model optimization	A real dataset may change the accuracy/latency trade-off.	Compare the tree with a quantized TinyML model using accuracy-per-byte and latency.
Stronger safety architecture	Industrial deployment requires higher fault tolerance.	Add redundant sensing, independent alarm hardware and authenticated firmware.
Energy validation	Current power values are estimates.	Measure current and tune sleep/radio duty cycle while preserving deadlines.

11. Overall Reflection

This assignment changed my understanding of embedded systems from simply interfacing sensors to engineering a complete real-time system. I learned that AI accuracy, RTOS scheduling, interrupt latency, memory use, communication and safety must be considered together. A model can be accurate but unsuitable if it misses deadlines, and a monitoring system is not robust if a network failure disables the local alarm path. The most valuable learning was connecting quantitative response-time analysis and resource budgets to concrete architecture decisions. If I continue the work, I would replace the synthetic assumptions with measured machine data, hardware WCET measurements and experimentally validated power and interrupt-latency results.

12. Submission Evidence Checklist

Required Evidence	Status
Individual Embedded C / FreeRTOS source	Prepared – attach source screenshot/file
RTOS task implementation	Prepared – task structure included
Scheduling calculations	Completed – included in technical report
SystemC model and simulation output	Prepared – source + reference trace; compile/run with SystemC toolchain
AI training/evaluation evidence	Completed – synthetic validation + confusion matrix
Interrupt implementation	Designed – ISR/semaphore path documented
Synchronization implementation	Designed – queue/mutex/semaphore/event-group mechanisms documented
Memory/resource analysis	Completed – budget tables included
Safety analysis	Completed – failure/mitigation matrix included
Energy comparison	Completed – illustrative 80 mA vs 55 mA design comparison
Screenshots/graphs/version history	Graphs included; add personal dated development/version evidence if available